

321-4002 – Software Engineering

Team Project for One or Two Member Teams

Delivery Deadline: 14/01/2026

GOALS

The main goal of this software project is to enable students to participate and receive relative experience in the cooperative development of software systems. A special emphasis is put on all the activities of the software development process and especially the requirements engineering, design, implementation and testing ones. In addition, an important attention is given to the use of particular technologies that promote cooperation, construction automation, dependency management and unit testing, such as Maven, JUnit (if Java is used) & Git. Finally, a special focus is made on the development of service-oriented systems, which include one or more RESTful services that can be developed through the assistance of a respective framework, like Jersey or Spring Boot.

Please note that some of the mentioned technologies (Maven, JUnit, Jersey) are language-specific. So, in case you decide to use a different implementation language, you need to adopt similar technologies (e.g., pip/Poetry/Pipenv, Django REST Framework, pytest for Python).

PROJECT TOPIC

FUNCTIONS / REQUIREMENTS

You need to develop the backend of a cinema management system by implementing RESTful web services that connect to an underlying database for managing all cinema-related entities. The core functionality will cover the management of programs (seasons) as well as the film screenings submitted to those programs. The scope ends at the final scheduling and public announcement of the program—no ticket sales or attendee registration are included. The system must support the full workflow from program creation, through screening submissions, staff assignment, reviews, and scheduling, up to the final publication and announcement of the lineup. Screening management must include creation, update, submission/withdrawal, handler assignment, review (score and comments), approval/rejection, final submission (freeze), and acceptance into the schedule, as well as search and role-aware view functions.

The program management system must include the following functions:

- *Program creation*: A program shall be created only if its name is unique and all its required fields are present. The system shall auto-generate program id and creation date and sets the program's state to CREATED. In this case, the creator (current user/requester) becomes PROGRAMMER of the program.
- *Program update*: It can be possible for a PROGRAMMER to modify a program's name, description, dates, and role assignments (PROGRAMMERS/STAFF). The program's creator cannot be removed from its PROGRAMMERS set. All updates must be completed before the program reaches the ANNOUNCED state.
- *Add programmers*: The system shall allow a PROGRAMMER the addition of registered users as PROGRAMMERS. If the user is already registered as a PROGRAMMER, return an appropriate error.
- *Add staff*: The system shall allow a PROGRAMMER the addition of registered users as STAFF. After the SUBMISSION state is reached for the program, the STAFF set is frozen (cannot be modified any more).

- *Program search*: It shall be possible to search programs by name, description, dates, film title, auditorium with AND semantics (when the previously mentioned parameters have values supplied to them, then they are AND combined meaning that all these conditions must be satisfied in order to return the respective program.). If no criteria are supplied, then all programs must be returned. In any case, the results must be filtered by role and then sorted first by date and then by name.
- *Program view*: It shall be possible to present the details of a program based on its id; these details (i.e., fields) must be redacted based on the current user role (e.g., Visitors see only public info).
- *Program deletion*: This function is allowed by any PROGRAMMER of the program only if that program is in the CREATED state.
- *Program State update*: This function allows PROGRAMMERS to update a program's state. Any unauthorized users that try to access this method, must have an error returned to them. States may not be rolled back (SUBMISSION → CREATED). Only the following program state transitions shall be allowed:
 - *Submission start* (CREATED → SUBMISSION): Film/screening submissions open and reviews are not allowed.
 - *Stage-manager assignment start* (SUBMISSION → ASSIGNMENT): Submissions close and the assignment of STAFF handlers begins.
 - *Review start* (ASSIGNMENT → REVIEW): Staff assignment is finished and screening reviews can proceed.
 - *Schedule making* (REVIEW → SCHEDULING): The reviews stop and only screening approval/rejection and tentative scheduling are allowed.
 - *Final publication start* (SCHEDULING → FINAL_PUBLICATION): Films/screenings submit final versions with any required changes (as demanded by the reviews).
 - *Decision making* (FINAL_PUBLICATION → DECISION): Make final accept/reject decisions. Any approved screening not finally submitted is automatically rejected.
 - *Program announcement* (DECISION → ANNOUNCED): The content is locked and ready for public publication.

The Screening management system must include the following functions:

- *Screening creation*: Create a screening (film + auditorium candidate). Only a USER may create a screening. On success, the system auto-assigns him/her the role of SUBMITTER while the screening state is CREATED. During creation the screening must be assigned to a program.
- *Screening update*: A SUBMITTER can update editable fields of a CREATED screening. While in the CREATED state a screening can be updated to have a specific Auditorium and include a specific a Film. Also a start time and end time can be specified.
- *Screening submission*: When a screening's information is complete, submit it to the program. On success, the screening's state changes from CREATED → SUBMITTED. Submission is accepted only if the program is in SUBMISSION and the screening is complete (film, auditorium candidate, derived end time via film duration). Function can be accessed only by a SUBMITTER.
- *Screening withdrawal*: A screening can be withdrawn by a SUBMITTER only if it is at CREATED state. On withdrawal, it is deleted since it has not entered the formal review.
- *Assign handler (STAFF)*: During ASSIGNMENT, a PROGRAMMER assigns exactly one STAFF member as the screening's handler.
- *Screening review*: The assigned STAFF provides a review (numeric score + detailed comments)

assessing technical/curation fit of the screening. Accepted only when the program is in REVIEW state.

- *Screening approval*: Mark as approved (pending scheduling). Allowed only in SCHEDULING by a SUBMITTER, may include conditional notes for final changes.
- *Screening rejection*:
 - *Manual by PROGRAMMER*:
 - In SCHEDULING, based on review/comments.
 - In DECISION, if the final submission fails to address required changes.
 - *Automatic by system*: In DECISION, any approved but not finally submitted screening is auto-rejected.

Rejection reason must be recorded in all cases.

- *Screening final submission*: Submit the final bundle of changes. Allowed only when the respective program is in FINAL_PUBLICATION. After success, details are frozen. Regular updates are allowed only when screening is in CREATED state. Function can only be accessed by a SUBMITTER.
- *Screening acceptance* (final scheduling): In DECISION (respective program's state), mark approved & finally submitted screening as SCHEDULED (final). Function can only be accessed by a PROGRAMMER.
- *Screening search*: Search screenings within a program by film title, cast, genre, and/or date range. Each filter is optional. When multiple filters are provided, the system applies AND semantics (all provided conditions must be satisfied). Within a single text filter (e.g., title, cast, genre), all words entered must appear in that field (e.g., title="star war" matches "Star Wars"). Matching is case-insensitive. If no filters are supplied, all screenings of the program are returned according to access rights. After role filtering, results are sorted by film genre, then film title; for timetable views, sort by start_time instead.
- *Screening view*: Present details by id; redact based on role.

Most of the system functions must be rapidly executed (at most 5-10 seconds of execution time per request/call). The system must also be reliable. This means that it should not exhibit internal errors while it needs to produce suitable error messages in case it is wrongly utilized by its users (e.g., wrong or invalid or incomplete input is supplied by the user in their request). This also means that the system must reliably avoid performing multiple times the same successfully executed function (e.g., double creation of a performance) unless the function is idempotent (i.e., it always produces the same result as, e.g., in the case of viewing a performance). Further, the system must be secure. This means that its users must be authenticated and mapped to suitable roles while the access to its functions must be subject to suitable access (control) rules. Additionally, the system should implement rate limiting to prevent abuse, especially for functions like performance submissions or festival searches. Finally, the system must support transactionality, especially with respect to the data that are stored in the underlying database. In other words, there must be consistency in the way the database is updated while this update should be successful only if the corresponding system function (e.g., screening scheduling) has been completely and properly executed. The system should also implement proper error handling and logging mechanisms to facilitate troubleshooting and maintain an audit trail of all significant actions, which is crucial for managing large-scale events like music festivals.

ROLES

There are 6 main user roles that need to be supported by the system:

- *VISITOR*: An anonymous (not authenticated) user. Visitors may search and view only announced programs and scheduled screenings. Viewing is restricted: they see a program's basic info (name, dates, auditorium, description, programmer names) and for screenings only film title, genre, scheduled time, and auditorium. No other management actions are allowed.

- *USER*: An authenticated user. Users can be assigned program-specific roles (PROGRAMMER or STAFF or SUBMITTER); otherwise, they have the same search/viewing rights as VISITOR.
- *PROGRAMMER*: An authenticated user who either created a program or was added as a programmer to it. Programmers can execute all program management actions, can approve/reject/accept screenings, and can assign handlers (STAFF) to screenings in that program. They can view all details of their programs and all screenings within them. For programs not owned or not part of their PROGRAMMER set, they have the visitor rights.
- *STAFF*: An authenticated user who can be assigned to handle one or more screenings in a specific program. A STAFF member reviews only the screenings they are assigned to and can see full details for those screenings; otherwise, they have VISITOR rights. This segregation prevents conflicts of interest.
- *SUBMITTER*: An authenticated user who can update own CREATED screenings, submit/withdraw own screenings, final-submit own approved screening, and view full details only of their own screenings.

All registered users start without a program-specific role. They may create a Program (season) in which case they immediately become that program's PROGRAMMER or they may submit films/screenings in a specific program (as owners of those submissions) in which case they immediately become that program's SUBMITTERS. While a program is being managed, other authenticated users can be added as STAFF for that program. A user may hold roles in multiple different programs, but within one program roles are strictly separated to prevent conflicts of interest, for example a user can be either PROGRAMMER or STAFF, never both. In addition, to preserve impartiality, a PROGRAMMER should not submit screenings in his/her own program. He/she should be only responsible for the management of that program. Each role carries clearly defined responsibilities and access limits to maintain the integrity of the program management process and avoid unfair advantage.

The following table summarizes the access rights of the five aforementioned roles:

Role	Allowed Functions	Comments
VISITOR	View/search announced programs and scheduled screenings	Sees limited info only (program basics; for screenings: film title, genre, time, auditorium).
USER	All VISITOR functions plus program create and screening create.	Sees limited info only (program basics; for screenings: film title, genre, time, auditorium). Also they can create a program or a screening.
PROGRAMMER	All USER functions plus full program management (for their programs): program create/update/delete, role assignments (add PROGRAMMER/STAFF), lifecycle transitions, screening approval/rejection/acceptance , STAFF handler assignment.	Can view and manage all details for programs they manage and can view all screenings within them. A PROGRAMMER should not submit screenings in his/her own program.
STAFF	All USER functions plus: review assigned screenings (score & comments).	Can view full details of screenings they're assigned to handle.
SUBMITTER	All USER functions plus update own CREATED screenings, submit/withdraw own screenings, final-submit own approved screenings, and view full details only of their own submissions/screenings.	Can view full details of screenings they have created.

ENTITIES

The system should be in the position to manage the following main entities:

- *Screening*: is characterized by a single identifier, which needs to be automatically produced by the system during its creation. The same holds for the screening's creation date, which also needs automatic production. A screening belongs to a specific program (season) and this linkage is obligatory. It must have a state, a start time (required at scheduling) and an end time (the end time is to be given manually and its difference with the start time must be greater or equal to the film duration). Film information (title, cast, genres and duration) and auditorium information (auditorium name), are stored within the screening record itself and must be provided at the workflow stages where they are required. A screening requires to have one user assigned to it (with role STAFF) as its handler (projection/stage manager). This assignment is conducted at a specific program state by calling a specific screening-management function so only then this association should be supplied (thus, it must be initially empty). Similarly, the reviewer comments and score can only be supplied during the screening's review. A screening must have a specific state at any time point. The possible states are the following: (a) CREATED, (b) SUBMITTED, (c) REVIEWED, (d) APPROVED, (e) SCHEDULED, and (f) REJECTED. Initially, the screening state is CREATED. If the screening gets successfully submitted, it moves into the SUBMITTED state. If the screening is reviewed (while the program is in the REVIEW phase), it transits to the REVIEWED state. If the screening is rejected (while in the, SCHEDULING, or DECISION program phases/states), it enters the REJECTED state, which is final. If the screening is approved (while in the SCHEDULING phase), it transits into the APPROVED state. Once being in APPROVED state, the screening can move into the SCHEDULED or REJECTED states depending on whether it is accepted into the timetable or rejected, respectively. The SCHEDULED and REJECTED states are final states.
- *Program (Cinema Season)*: is characterized by a single identifier, which needs to be automatically produced by the system during its creation. The same holds for the program's creation date, which also needs automatic production. A program has a unique name, description and dates, which are considered as obligatory information, along with at least one PROGRAMMER (an authenticated user) that must be assigned at creation time. A program needs to be associated with its screenings and can be associated with multiple PROGRAMMERS and STAFF members, if needed. Finally, a program should be associated with one or more STAFF members (i.e., authenticated users). The STAFF members set should be finalized before the handler assignment activity. Thus, after the SUBMISSION state of the program, no changes to the STAFF members set are allowed. A program must have a specific state at any time point. The possible states are the following: (a) CREATED, (b) SUBMISSION, (c) ASSIGNMENT, (d) REVIEW, (e) SCHEDULING, (f) FINAL_SUBMISSION, (g) DECISION and (h) ANNOUNCED. Initially, a newly created program is at the CREATED state. Then, it can transit to the ANNOUNCED state, i.e., the final one, sequentially in a manual manner through the execution of respective program-management functions. The program moves from CREATED to SUBMISSION by executing the *submission start* function. Once this is done, screenings can be submitted but not reviewed. The program moves from SUBMISSION to ASSIGNMENT by executing the *handler assignment start* function. This indicates that screenings cannot be submitted anymore and only staff assignments are allowed. The program transits from ASSIGNMENT to REVIEW by executing the *review start* function. This signifies that staff assignment has been finished and screening reviewing can proceed. The program moves from REVIEW to SCHEDULING by executing the *schedule-making* function. This indicates that no review is then possible and only scheduling of screenings is allowed (i.e., approval or rejection of screenings). The program moves from SCHEDULING to FINAL_SUBMISSION by executing the final submission start function. During this state, submitters can submit final versions of their approved screenings, including any required changes based on reviews. The program moves from FINAL_SUBMISSION to DECISION by executing the *decision-making* function. During this state, final decisions (either accept or reject) are made on the (finally) submitted screenings. Any approved screenings that were not finally submitted by this point

automatically become REJECTED. Finally, the program moves from DECISION to ANNOUNCED by executing the *announcement* function. No more changes to the program's lineup or schedule are allowed in this case (all program information is frozen).

- *User*: must have a username and password based on which their authentication can be performed. Further, they must have a (full) name. Finally, they can have different roles in different programs (e.g., PROGRAMMER in one program and STAFF in another one) but at most one role in a specific program. *Please note that the cinema management backend will mainly read user information for user authentication and authorisation purposes and will only modify it for supporting the management of program-specific roles (see also next assumption/constraint).*

ASSUMPTIONS/CONSTRAINTS

To facilitate the project implementation, it can be assumed that:

- There exists a *user management system* responsible for managing all users, irrespective of their roles in the system. As such, user authentication can be realized by utilizing the shared, underlying database (between the user and film management systems) where it is checked whether the current user has supplied a correct and valid username and password combination. *Both user authentication and authorization are functions that must be supported by the cinema management backend.* In this way, it is considered that they are not supplied as services by the user management system.

First Project Part (15/100)

Based on the above analysis of the project high-level, user requirements, you must produce a set of detailed system requirements, both functional and non-functional. You are free to make any assumption in case you regard that the above analysis is vague or ambiguous, while producing the needed system requirements. All generated requirements need to be incorporated in the respective part of the report to be delivered. You can use natural or structured natural language to specify the needed requirements.

Second Project Part (30/100)

Based on the extracted system requirements, you should now proceed to system design. In this case, you need to produce and incorporate, at the appropriate place in the report to be delivered, the following diagrams:

- A context diagram for modelling the environment that surrounds the software system to be developed
- Use case diagrams to model the main use cases of the system
- A component diagram to model the system architecture
- A set of activity diagrams in order to detail the expected behavior in the context of some of the modelled use cases
- A set of sequence diagrams in order to detail the expected behavior in the context of some of the modelled use cases. Please note that the set of activity and sequence diagrams must jointly cover all the use cases modelled for the system. Further, there should not be a pair of activity and sequence diagrams that detail the same use case.
- An Entity-Relationship (ER) diagram or a database diagram or a class diagram to facilitate the design and implementation of the underlying database. In the case of a class diagram, the focus should be on the main informational entities of the system and their relations. Further, methods should not be modelled in classes.

Every incorporated diagram must be accompanied with its short textual analysis / explanation within the report to be delivered.

Third Project Part (50/100)

Based on the system models produced in the second project part, proceed with the collaborative implementation of the system by utilizing any programming language you prefer. However, the use of the Java programming language is recommended due to its respective technologies that have been taught and exercised during the laboratorial part of the course. The main system functions must be supplied in the form of one or more RESTful services. To this end, a suitable framework for developing RESTful services must be adopted (such as Jersey or Spring Boot if the implementation language is Java).

The collaborative implementation of the system must be realised by using Git and a private repository in Github. In the private repository, the contributing members should be the members of the team and the course instructors, who need to be invited (user names: kkritikos & fkatsifas). Apart from the basic system source code, unit testing code has also to be implemented (by utilising a respective framework like JUnit). Finally, the use of Maven is suggested, in case Java is used as an implementation language, due to this ability to automatically manage the code dependencies and to automatically produce the artifacts for the various related software construction activities (e.g., main and test code packages, test reports, etc.). In case of using any other programming language apart from Java, it is recommended that any corresponding framework (with respect to Maven) should be adopted.

The main deliveries for this project part must be the following:

- The repository URL of the implemented system in Github
- Implementation documentation: what is the project structure, a class diagram, analysis of main system classes, instructions on how to compile, package and execute the system code, which are the prerequisite systems/software packages (e.g., web/servlet containers, database servers) that must be already installed in the underlying operating system, SQL scripts for database/table generation.
- Test documentation: on which system parts did the testing focus, which test cases were covered, special preconditions or dependencies that are required for executing the unit tests

All these deliveries must be incorporated in the right place of the report to be delivered.

REPORT (5/100)

Apart from the aforementioned project parts with specific deliveries to be incorporated in the final project report to deliver, this report must also include additional parts, which will be graded with 5/100.

The report structure in terms of its main chapters/units must be the following:

1. *Cover Page*
2. *Summary*: must summarise the main functions and features of the system under development and the respective system environment
3. *Organisation*: you need to stress the following:
 - a. Which were the basic roles in the development and which members of the team played what roles
 - b. The basic development activities/stages conducted/followed (obviously, they map to the first three project parts) as well as the activities duration and total project development time (in hours)

4. *Requirements* (first project part): you must organise this chapter into two sections, one section for the functional and one for the non-functional requirements. The requirements should be numbered in a hierarchical manner in each section.
5. *Design* (second project part): this chapter must be organized into the following sections:
 - a. *System Architecture and Context*: supply and shortly explain the component diagram and context diagram of the system to be developed
 - b. *Use Cases*: supply and shortly explain the use cases (diagrams) for the system to be developed
 - c. *System Behaviour*: supply and shortly explain activity and sequence diagrams, detailing the user-system interactions and system behaviour in the context of the use cases to be supported
 - d. *System Entities*: supply and shortly explain the system's ER / database diagram
6. *Implementation* (third project part): this chapter must be organized into the following sections:
 - a. *Github*. Project repository URL in GitHub και contributors. Special repository settings (if applicable). Flow of working in Git and basic development branches during system implementation
 - b. *Implementation Documentation*
 - c. *Test Documentation*
7. *Conclusions*:
 - a. The experience attained from this project
 - b. The problems and obstacles that were faced
 - c. Best practices that were followed or had to be followed, which could be (also) applied in your future (software development) projects

SUBMISSION INSTRUCTIONS

One of the two members (or the sole member) of the team must submit the final report in the relative project of the course's eclass (deadline: until 14/01/2026). The report format can be WORD (.doc(x)) or PDF (.pdf). The name of the report file to be submitted must follow the following pattern: <AM1>_<AM2>_report.<ending> where <AM1>, <AM2> are the matriculation numbers of the two team members, respectively and <ending> is the report file's ending based on its format. In case of a one-member team, the file name pattern is simplified as follows: <AM1>_report.<ending>.

The project will be remotely examined via Zoom. Instructions for this examination will be supplied via a related announcement to be posted in the course's eclass.